

AstroBiz OS — Frontend Architecture Proposal

Prepared for the desktop Tauri frontend prototype.

Review summary

I reviewed the full specification in docs/Frontend_SPEC.md and checked the repository state. The workspace currently contains only the project README and the specification document. There is no existing Tauri app, JavaScript frontend scaffold, or package manifest yet. Nothing should be deleted because this is a clean starting point.

Recommended frontend structure

Because the project is early and uses mock data first, I recommend a lightweight modular vanilla JavaScript architecture for the desktop shell rather than a heavy framework. This keeps the Tauri app simple while still enforcing clean separation between UI, state, repositories, and data.

- apps/desktop/src/app - application shell, entry point, router, app layout
- apps/desktop/src/core - config, constants, utilities, app-wide helpers
- apps/desktop/src/shared - reusable UI components, style tokens, state hooks, providers
- apps/desktop/src/state - application state, actions, selectors, reducers
- apps/desktop/src/models - domain data contracts such as Customer, Appointment, Invoice, Kundali
- apps/desktop/src/repositories - repository interfaces and mock or API implementations
- apps/desktop/src/data/mock - seed data for the prototype
- apps/desktop/src/features - feature modules for auth, dashboard, customers, billing, CRM, settings, and more
- apps/desktop/src/services - API transport, notifications, validation, formatting, utilities

Architecture recommendation

I recommend a simple architecture centered on five layers: UI, state/controller, repository, mock data, and services/utilities. The app should render views from the UI layer, delegate actions to a controller or store, call a repository abstraction, and only then rely on mock seed data or fallback API adapters.

This matches the spec's desired flow: UI → Controller / State → Repository → Mock Data, and later: UI → Controller / State → Repository → REST API → Backend.

Reusable UI components

The spec requires reusable, professional SaaS components. These should be created early and shared across modules: Sidebar, Topbar, PageHeader, MetricCard, StatusBadge, EmptyState, LoadingState, ErrorState, Input, Select, Modal, DataTable, ConfirmationDialog, Toast, CustomerSummaryCard, AppointmentCard, InvoiceSummary, FollowUpItem, TimelineItem, and QuickActionButton.

State, data, and repository structure

Use a single application store with slices such as auth, dashboard, customers, kundali, appointments, services, billing, crm, reports, and settings. Each slice should hold records, selected item state, loading flags, error state, filters, and pagination. Models should be pure data contracts, while repositories provide access to crud operations and list retrieval.

Define repository interfaces first, then implement mock repositories for the prototype. The UI and state logic should depend on the interface and not on a concrete implementation.

How mock repositories will later be replaced

The replacement should happen through a repository factory or dependency injection layer. In prototype mode, the app resolves MockCustomerRepository, MockAppointmentRepository, and similar classes. When the backend is ready, the same application wiring resolves ApiCustomerRepository and ApiAppointmentRepository instead. The screens continue to call the same methods, so the UI remains stable.

Recommended next step

The next phase should be: create the desktop app shell, add the shared design system and base components, then build the dashboard using mock data, followed by customers, customer profile, and the rest of the order listed in the spec. I have not implemented modules yet, and I am waiting for approval before creating the frontend scaffold and moving into the desktop project structure.